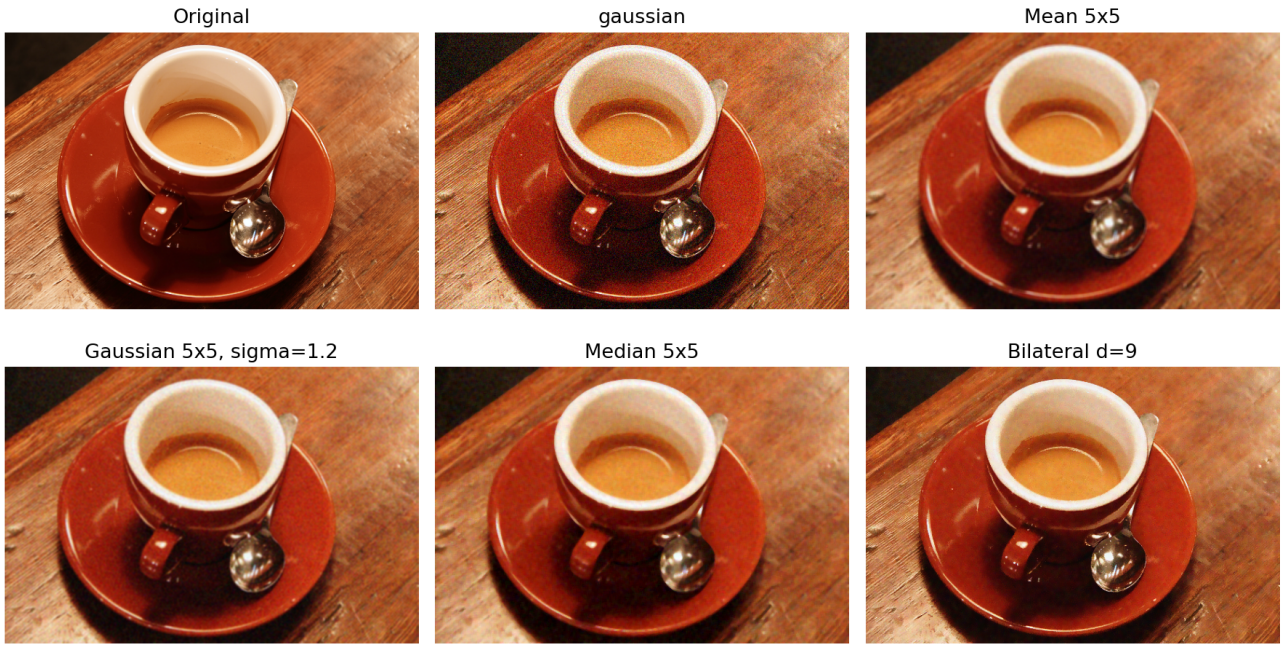
Lab 1
Image Enhancement and Filtering

Image processing methods
Academy of Technology and Art (ATA)
Instructor: Kamil Musiał, PhD Eng.



Learning objective

Learn how to improve contrast and reduce noise. Compare filters using the same data, analyze histograms and assess results numerically. This material develops the scope of Lab 1 in the supplied screenshot.

The package includes **lab1.ipynb**, **lab1.py**, requirements, photographs and actual example results. Suggested working time: 90-120 minutes. The detailed tasks are a proposed teaching implementation rather than an official instructor-issued handout or grading scheme.

1. Prepare the environment

Extract the entire ZIP. Open a terminal in the directory containing lab1.py and the images folder. Python 3.12 is recommended. Internet access is needed to install packages; the exercise photographs are already included.

```
python -m venv .venv

# Windows cmd
.venv\Scripts\activate.bat
# Windows PowerShell
.venv\Scripts\Activate.ps1
# macOS / Linux
source .venv/bin/activate
```

Choose the activation command for your operating system. If PowerShell blocks activation, use .venv\Scripts\python.exe directly in place of python.

Install and run

```
python -m pip install -r requirements.txt

# Complete script
python lab1.py --out results_new

# Notebook
python -m pip install jupyterlab
python -m jupyter lab
```

Open lab1.ipynb in Jupyter and run the cells from top to bottom. The notebook interpreter must use the environment in which the packages were installed. If needed, install ipykernel and register the environment.

```
python -m pip install ipykernel
python -m ipykernel install --user --name image-lab
# Then select the image-lab kernel in Jupyter.
```

Headless OpenCV does not open cv2.imshow windows. The exercise uses Matplotlib and PNG output. PyTorch appears in the broader tool list in the supplied brief, but is not required for these classical operations.

2. Read and represent an image

OpenCV normally loads BGR channel order; Matplotlib expects RGB. The script provides read_rgb to load a file and explicitly convert the channels. imdecode also supports Unicode paths on Windows.

```
from lab1 import read_rgb, save_image
import cv2
import numpy as np
import matplotlib.pyplot as plt

rgb = read_rgb("images/coffee.png")
gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
print(rgb.shape, rgb.dtype, rgb.min(), rgb.max())
plt.imshow(rgb)
plt.axis("off")
```

Representation	Shape	Meaning
RGB	H × W × 3	R, G, B
Gray	H × W	One intensity channel
uint8	0-255	Photograph encoding in this lab

coffee has shape (400, 600, 3). image[y, x] selects a pixel in row y and column x. Do not reverse these indices. Convert to float before calculating differences, squares or brightness changes.

Check a save-and-read round trip

```
save_image("notebook_results/copy.png", rgb)
copy = read_rgb("notebook_results/copy.png")
print(np.array_equal(rgb, copy))  # True for PNG
```

PNG stores the samples without lossy compression. JPEG is useful for photographs but changes sample values and can complicate the evaluation of small differences between methods.

3. Color models in practice

Goal	Choice	Why
Display a photograph	RGB / sRGB	Screen representation
Printing	CMYK + ICC	Ink separation and print conditions
Color mask	HSV	Convenient hue thresholds
Color contrast	Lab / YCrCb	Separate brightness
Basic histogram	Gray	One channel

```
hsv = cv2.cvtColor(rgb, cv2.COLOR_RGB2HSV)
lab = cv2.cvtColor(rgb, cv2.COLOR_RGB2LAB)
ycrcb = cv2.cvtColor(rgb, cv2.COLOR_RGB2YCrCb)
```

Standard OpenCV uint8 HSV uses H in 0-179 and S/V in 0-255. Red wraps around the ends of the hue range, so red segmentation often needs two masks. Hue becomes unreliable at low saturation.

Lab separates lightness L from the chromatic a and b components. OpenCV uint8 scales L to 0-255 and offsets a/b by 128. OpenCV RGB-to-Gray uses 0.299R + 0.587G + 0.114B. This practical conversion is not an exact measurement of physical luminance.

Mini-exercise: an HSV threshold

```
mask = cv2.inRange(hsv, np.array([0, 100, 80]),
                   np.array([25, 255, 255]))
selected = cv2.bitwise_and(rgb, rgb, mask=mask)
```

Inspect which coffee regions pass the threshold. Explain why color alone cannot recognize a particular object: the wood and parts of the cup may have similar colors. Lighting and white balance affect the result.

The presentation uses ideal RGB/CMYK formulas to teach the model. Actual printing requires an ICC profile appropriate to the printer and paper. OpenCV does not replace a complete color-management system.

4. Simulate two noise models

Here the clean original is known. Add noise and compare each denoised result with that reference. A naturally noisy photograph usually has no known clean version, so this full-reference evaluation is not directly available.

Gaussian noise

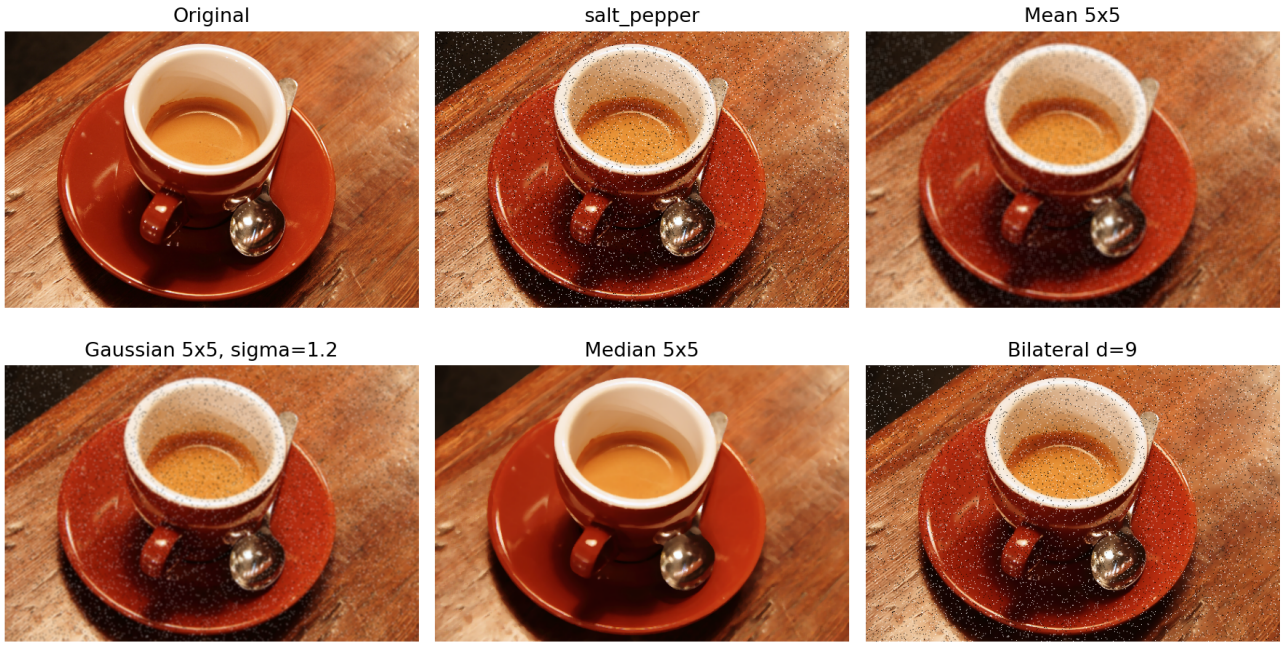
```
from lab1 import gaussian_noise, salt_pepper_noise
gauss = gaussian_noise(rgb, sigma=20, seed=42)
```

Each channel receives independent samples with mean 0 and standard deviation 20 on the 0-255 scale. Clip and round the result. Sigma=20 is approximately 0.0784 on a 0-1 scale. Clipping near the endpoints can change the noise distribution.

Salt and pepper

```
sp = salt_pepper_noise(rgb, amount=0.05, seed=42)
```

A random mask replaces complete pixels with black or white. Amount=0.05 means an expected total of 5%, approximately 2.5% of each type. The actual number of impulses can differ slightly from that expectation.



Fair comparison requires every filter to receive the exact same noisy array. Drawing new noise for each filter adds another variable. Keep the original unchanged.

5. Filters and their parameters

Method	Baseline settings	Expected effect
Mean	5×5	Smoothing and blurred edges
Gaussian	5×5, sigma=1.2	Smooth local averaging
Median	5×5	Resistance to impulses
Bilateral	d=9, sigmaColor=50, sigmaSpace=5	Preserve strong edges
NLM	h=8, hColor=8, windows 7 and 21	Compare similar patches

```
mean = cv2.blur(gauss, (5, 5))
gau = cv2.GaussianBlur(gauss, (5, 5), 1.2)
med = cv2.medianBlur(gauss, 5)
bil = cv2.bilateralFilter(gauss, 9, 50, 5)

from lab1 import filter_bank
filtered = filter_bank(gauss)
```

The complete filter_bank function explicitly uses BORDER_REFLECT_101 for mean, Gaussian and bilateral filtering. Median and NLM use their own boundary handling. Compare interior crops as well, where the border has no influence.

Mean filtering assigns equal weights. Gaussian filtering gives greater weights near the center. The median selects the middle ordered value, making it less sensitive to an isolated impulse. For color, the median operates separately on each channel.

Bilateral filtering considers spatial distance and value similarity. sigmaColor uses the data value scale; sigmaSpace uses pixels. Colored NLM expects BGR: the supplied wrapper converts before and after the operation.

NLM h=8 is conservative for noise sigma=20. Compare h=8,12,16. Higher h may remove more noise but also more texture.

6. Histograms and tone transforms

```
hist = np.bincount(gray.ravel(), minlength=256)
plt.plot(np.arange(256), hist)
plt.xlim(0, 255)
```

The horizontal axis is intensity; the vertical axis is pixel count. Counts sum to H×W. A histogram does not store positions: shuffling pixels changes the image but preserves the histogram. There is no single ideal histogram shape for every scene.

Contrast stretching

```
from lab1 import percentile_stretch, gamma_transform
stretched = percentile_stretch(gray, low=2, high=98)
```

Map p2 to 0 and p98 to 255; clip samples outside this interval. This reduces the influence of isolated extremes compared with min-max stretching. Ties can make the clipped fraction differ from 4%. The function handles constant images to avoid division by zero.

Gamma and a linear transform

```
gamma = gamma_transform(gray, gamma=0.6)
linear = np.clip(1.2 * gray.astype(np.float32) + 10,
                 0, 255).astype(np.uint8)
```

Our convention is Iout=255·(Iin/255)^gamma. Gamma below 1 brightens midtones. Some references use 1/gamma as the exponent; always state the formula. Tone adjustment is different from decoding sRGB.

Alpha changes the slope and contrast; beta shifts brightness. Brightening cannot recover detail already clipped in highlights or shadows. convertScaleAbs also takes an absolute value, so it is not always equivalent to a simple clip.

7. Equalization and CLAHE

```
equal = cv2.equalizeHist(gray)
operator = cv2.createCLAHE(clipLimit=2.0,
                           tileGridSize=(8, 8))
local = operator.apply(gray)
```

Global equalization uses the cumulative histogram to define one nonlinear mapping. A discrete output need not have a flat histogram. Equalization can increase visible noise and change the natural appearance of a scene.

CLAHE divides the image into regions, limits large histogram values and interpolates between results. TileGridSize=(8,8) is the number of regions along each dimension, not tiles of 8 pixels. A higher clipLimit generally permits stronger contrast enhancement.



Color version

```
from lab1 import clahe_rgb
color_result = clahe_rgb(rgb, clip_limit=2.0, grid=(8,8))
```

This function processes L in Lab and retains a/b before the return conversion. It limits independent RGB ratio changes, although small color shifts and gamut clipping can remain. Compare it with independent equalization of the RGB channels.

8. Sharpening and operation order

Unsharp masking increases the local contrast of fine structures by adding the difference between the image and a blurred version. It does not automatically recover information lost through severe blur or compression.

```
from lab1 import unsharp
sharp = unsharp(rgb, sigma=1.2, amount=1.0)
sharp_noise = unsharp(gauss, sigma=1.2, amount=1.0)
sharp_after = unsharp(filtered["bilateral"])
```

Formula: Isharp = I + amount·(I - blur(I)). Sigma controls the blur scale; amount controls the strength of the added detail. Excessive values create halos and amplify noise. Clipping to 0-255 can discard endpoint information.



Sharpened Gaussian-noise image: the disturbance also gains contrast.

Questions for the comparison

Is the cup edge clearer? Does the wood texture look natural? Are there bright halos? Does noise become more visible in dark regions? Inspect the identical crop at the same magnification.

A useful starting order is noise reduction, contrast enhancement and careful sharpening. It is not universal. Evaluate the entire pipeline for its goal, such as document readability or segmentation.

9. Metrics and timing

```
from skimage.metrics import peak_signal_noise_ratio
from skimage.metrics import structural_similarity

mse = np.mean((rgb.astype(np.float64) -
               med.astype(np.float64)) ** 2)
psnr = peak_signal_noise_ratio(rgb, med, data_range=255)
ssim = structural_similarity(rgb, med, data_range=255,
                             channel_axis=-1)
```

MSE is the mean squared difference. Lower is better for reference fidelity. **PSNR** = 10·log10(MAX²/MSE), with MAX=255 for uint8. Identical images have MSE=0 and infinite PSNR. For float 0-1, MAX is 1.

SSIM compares local structure, contrast and brightness. A value of 1 indicates agreement in the chosen configuration; negative values are possible. The default skimage window needs at least 7×7 pixels. channel_axis=-1 identifies RGB channels.

Compared images must match in size, alignment and range. Higher PSNR/SSIM does not guarantee the best OCR or the most attractive photograph. Contrast adjustment intentionally changes values and should not be judged only by fidelity to the original.

Execution time

```
import time
cv2.setNumThreads(1)
cv2.GaussianBlur(gauss, (5,5), 1.2)  # warm-up
samples = []
for _ in range(3):
    start = time.perf_counter()
    cv2.GaussianBlur(gauss, (5,5), 1.2)
    samples.append((time.perf_counter()-start)*1000)
print("Median [ms]:", np.median(samples))
```

Time only the filter, excluding file output, plotting and metric computation. Record hardware, image size, thread count and repetitions. Example CSV timings are not universal benchmarks.

10. Actual baseline results

coffee photograph, 600×400, RGB uint8. Seed=42. Gaussian sigma=20; salt-and-pepper amount=0.05. Every method receives the same noisy image for a given noise model.

Gaussian noise

Method	PSNR [dB]	SSIM
No filter	22.60	0.405
Mean 5×5	25.71	0.682
Gaussian 5×5	26.92	0.712
Median 5×5	25.99	0.671
Bilateral	29.25	0.728
NLM h=8	25.36	0.594

Salt and pepper

Method	PSNR [dB]	SSIM
No filter	17.60	0.380
Mean 5×5	24.72	0.593
Gaussian 5×5	25.25	0.597
Median 5×5	26.72	0.791
Bilateral	17.56	0.337
NLM h=8	19.20	0.413

With these settings, bilateral gives the best Gaussian-noise result. Median 5×5 leads the impulse-noise filter comparison. In a separate median window sweep, 3×3 reaches SSIM about 0.881. The larger windows do not provide a better trade-off here.

These results use one image and fixed settings without full optimization. NLM h=8 is conservative for sigma=20. General conclusions require more images, noise levels and repeated seeds.

11. Experiments and the report

Series	Changed parameter	Keep fixed
Gaussian	sigma: 10, 20, 30	Image, seed, filters
Impulses	amount: 0.01 / 0.05 / 0.10	Image, seed
Median	Window: 3, 5, 7, 9	Same sp array
Bilateral	sigmaColor: 25, 50, 75	d=9, sigmaSpace=5
CLAHE	clipLimit: 1, 2, 4	8×8 grid
NLM	h: 8, 12, 16	Windows 7 and 21

Change one factor per series. Extension: repeat one selected method for seed=1,42,100 and a second photograph. Report the mean and standard deviation of the metrics. Assess a naturally noisy image without a clean reference visually and through task performance.

Run on a second photograph

```
# Enter this command on one line:
python lab1.py --image images/chelsea.png --sigma 30 --out results_cat
```

Use a separate directory for each experiment. The script limits large custom photographs to 1000 pixels in width. In that case, the resized image becomes the reference.

Suggested report structure

1. Objective and data. 2. Environment and versions. 3. Noise and filter settings. 4. Full-image and identical-crop comparisons. 5. Histograms before and after transforms. 6. PSNR, SSIM and timing tables. 7. Conclusions with limitations. 8. Code or notebook.

A precise conclusion: “For coffee with amount=0.05, median 3×3 preserved more texture than 9×9 and achieved a higher SSIM.” An overly broad conclusion: “The median is always best.” Include a case where the chosen method makes the result worse.

12. Troubleshooting, files and sources

Problem	What to check
Missing package	Interpreter / kernel and python -m pip
Unexpected colors	Explicit BGR / RGB order
Incorrect MSE	Convert to float before subtraction
CLAHE error	One channel, normally uint8
SSIM error	Size, channel_axis, data_range
Cannot read image	Working directory and full path

The script and notebook were executed with Python 3.12.14, OpenCV 5.0.0, NumPy 2.3.5, scikit-image 0.26.0 and Matplotlib 3.10.8. requirements.txt pins the versions used for the example results. The basic operations also exist in OpenCV 4.x.

Output files

results contains individual PNG files, comparison panels, histograms.png, metrics.csv and summary.json. CSV records MSE, PSNR, SSIM and median filter runtime. JSON records parameters, versions and complete histograms. The notebook includes an executed example.

Primary documentation

OpenCV: filters

OpenCV: equalization and CLAHE

OpenCV: color conversions

scikit-image: metrics

scikit-image: data types

scikit-image: photograph provenance

ICC: color profiles

Photographs: coffee, Rachel Michetti, CC0, courtesy of Pikolo Espresso Bar. chelsea, Stefan van der Walt, CC0. camera, Lav Varshney, CC0. astronaut, NASA, public domain. Instructor details and the screenshot were supplied by the requester. Sources checked on 9 October 2026.